LARAVEL STUDENT LAB GUIDE

From Composer project creation to database setup and displaying records

Scope: Classroom walkthrough covering Laravel setup, MVC basics, migrations, models, seeders, and database records displayed in a Blade view (through the requested Module 7 outcome). This is a standalone practice guide; it does not claim to reproduce an unseen official Module 7 handout.

Environment: Windows + VS Code + PowerShell + PHP 8.2 or newer compatible version + Composer + Laravel 12. SQLite is used so students do not need to configure MySQL for this exercise.

Target output: A page at http://127.0.0.1:8000/tasks showing three records: Buy groceries, Submit report, and Fix login bug.

1 | Check your tools

Open VS Code > Terminal > New Terminal. Run these commands one at a time:

```
php -v
composer --version
```

Both commands must display version information. If either command is not recognized, ask your instructor to check the installation and PATH for that tool before continuing.

NOTE: You do NOT need to run composer global require laravel/installer, laravel new, or composer global config bin-dir --absolute for this method.

2 | Create a new Laravel project using Composer

Choose a writable project location. The example below uses Documents. If your instructor requires XAMPP htdocs, you may use C:\xampp\htdocs instead.

```
cd $HOME\Documents
composer create-project laravel/laravel LaravelStudentLab "^12.0"
```

Wait until Composer finishes and the PowerShell prompt returns. This command creates a new LaravelStudentLab folder. Do not run it again with the same folder name.

```
cd LaravelStudentLab
php artisan --version
```

Expected: a Laravel Framework 12.x version. If Composer reports a missing PHP extension or dependency problem, read the specific error before changing anything.

3 | Configure the SQLite database

In VS Code Explorer, open the project root .env file. Find or set:

```
DB_CONNECTION=sqlite
```

If .env contains DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, or DB_PASSWORD settings for MySQL, they are not needed for this SQLite exercise; you may leave them commented out. Save the file.

Check whether the SQLite file already exists:

```
Test-Path .\database\database.sqlite
```

If the result is False, create the file:

```
New-Item -Path .\database\database.sqlite -ItemType File
```

If the result is True, do not create it again. Ensure the pdo_sqlite and sqlite3 PHP extensions are enabled if a driver error appears.

4 | Create the tasks table with a migration

In the project root terminal:

```
php artisan make:migration create_tasks_table --create=tasks
```

Open the newly created file inside database/migrations (its filename begins with a timestamp and ends with create_tasks_table.php). In the up() method, use:

```
public function up(): void
{
    Schema::create('tasks', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->string('status')->default('pending');
        $table->timestamps();
    });
}
```

Keep the generated down() method, which drops the tasks table. Save the file, then run:

```
php artisan migrate
```

Expected: migration completed without errors. This creates the tasks table in SQLite.

5 | Create the Eloquent Task model

```
php artisan make:model Task
```

Open app/Models/Task.php and replace its contents with:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Task extends Model
{
    protected $fillable = ['title', 'status'];
}
```

Explanation: The Task model represents the tasks table. The $fillable property permits mass assignment of the title and status fields, including when using Task::create().

6 | Insert sample records with a seeder

```
php artisan make:seeder TaskSeeder
```

Open database/seeders/TaskSeeder.php and replace its contents with:

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
```

```
use App\Models\Task;

class TaskSeeder extends Seeder
{
    public function run(): void
    {
        Task::create(['title' => 'Buy groceries',
                      'status' => 'pending']);
        Task::create(['title' => 'Submit report',
                      'status' => 'pending']);
        Task::create(['title' => 'Fix login bug',
                      'status' => 'done']);
    }
}
```

Save, then execute:

```
php artisan db:seed --class=TaskSeeder
```

NOTE: Run this seeder only once for this exercise. Running it again inserts another set of three records, so duplicates may appear. The import use App\Models\Task; is required.

7 | Create a controller to retrieve the records

```
php artisan make:controller TaskController
```

Open app/Http/Controllers/TaskController.php and replace its contents with:

```
<?php

namespace App\Http\Controllers;

use App\Models\Task;
use Illuminate\View\View;

class TaskController extends Controller
{
    public function index(): View
    {
        $tasks = Task::all();

        return view('tasks.index', ['tasks' => $tasks]);
    }
}
```

Explanation: Task::all() reads all task records. view('tasks.index', ...) passes the records to the Blade page.

8 | Register the /tasks route

Open routes/web.php. Keep any existing welcome route. Add the controller import and the route:

```
use App\Http\Controllers\TaskController;

Route::get('/tasks', [TaskController::class, 'index']);
```

NOTE: Put the use statement with the other imports near the top of the file, and the Route::get statement with the routes. Do not paste either inside another route callback.

9 | Create the Blade page that displays records

Inside resources/views, create a folder named tasks. Inside it, create index.blade.php. Paste:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My Tasks</title>
    <style>
        body { font-family: Arial, sans-serif;
               max-width: 850px; margin: 40px auto; }
        table { width: 100%; border-collapse: collapse; }
        th, td { padding: 10px; border: 1px solid #ccc;
                 text-align: left; }
        th { background: #e9f1f2; }
    </style>
</head>
<body>
    <h1>My Tasks</h1>
    <table>
        <thead>
            <tr><th>ID</th><th>Task Title</th><th>Status</th></tr>
        </thead>
        <tbody>
            @forelse ($tasks as $task)
                <tr>
                    <td>{{ $task->id }}</td>
                    <td>{{ $task->title }}</td>
                    <td>{{ ucfirst($task->status) }}</td>
                </tr>
            @empty
                <tr><td colspan="3">No tasks found.</td></tr>
            @endforelse
        </tbody>
    </table>
</body>
</html>
```

Blade uses @forelse to loop through the records, and {{ ... }} to display values safely.

10 | Run the application and check the output

From the LaravelStudentLab project directory, run:

```
php artisan serve
```

Open http://127.0.0.1:8000/tasks in your browser (or use the URL shown by Artisan if port 8000 is unavailable).

Expected output

ID	Task Title	Status
1	Buy groceries	Pending
2	Submit report	Pending
3	Fix login bug	Done

IDs are usually 1, 2, and 3 in a fresh database; actual IDs may differ if records were inserted previously.

11 | Quick troubleshooting reference

laravel is not recognized

This guide does not use laravel new. Use composer create-project instead. It does not repair the laravel command itself.

composer is not recognized

Check Composer installation and the PATH used by the current terminal. Reopen VS Code after PATH changes.

php is not recognized

Check PHP installation and PHP PATH, then verify php -v.

Could not find driver (SQLite)

Enable the appropriate PHP SQLite extensions in the php.ini used by CLI PHP; restart terminal and retry.

Target class [TaskController] does not exist

Ensure routes/web.php imports App\Http\Controllers\TaskController and the class/file name matches.

Class "Database\Seeders\Task" not found

Add use App\Models\Task; at the top of TaskSeeder.php.

MassAssignmentException

Check Task.php contains protected $fillable = ['title', 'status'];

No tasks found

Check DB_CONNECTION=sqlite, migration ran, and TaskSeeder ran. Use php artisan tinker, then App\Models\Task::count();

Failed to download / Permission denied

Wait for Composer to finish; inspect the last error. Check network, writable folder, security restrictions, and free disk space. Do not assume PATH is the issue.

Duplicate rows

TaskSeeder was probably executed more than once. Avoid rerunning it; ask your instructor before resetting the database.

12 | Student submission checklist

Submit screenshots showing: (1) successful Composer project installation or php artisan --version, (2) successful migration, (3) successful seeding, (4) the /tasks page displaying three records. Include the project folder name and the code files: Task.php, TaskSeeder.php, TaskController.php, routes/web.php, and tasks/index.blade.php.

Reflection questions: What is the role of the migration? What is the difference between the model and controller? How does the Blade view receive task records? Why do we use a seeder?

End of guide.